

Military Tank Detection & Tracking

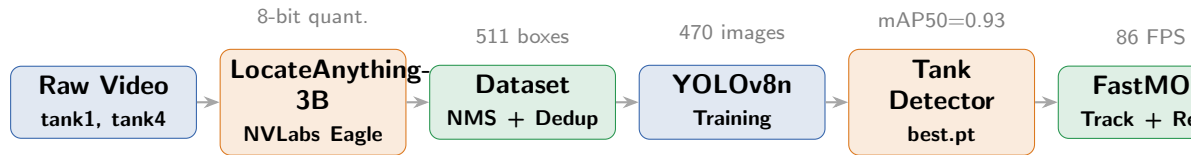
From Raw Video to Multi-Object Tracking

June 7, 2026

Outline

- 1 System Overview
- 2 Data & Annotation
- 3 YOLOv8 Training
- 4 FastMOT Integration
- 5 Results

Complete Pipeline Overview



Annotation

Open-vocabulary VLM auto-labels tanks with no manual work

Tracking

Kalman filter + KLT optical flow + PyTorch ReID

Input Data

Video Sources

- tank1.mp4 — 417 frames, 768×432 , 24 fps
Side-view US Abrams, desert terrain
- tank4.mp4 — 722 frames, 768×432 , 30 fps
Aerial/drone view, multiple vehicles

Challenge

Large domain gap between videos. Aerial footage has vastly different appearance from side-view footage.

Domain Distribution

Video	View	# Vehicles
tank1	Side-angle	1
tank4	Top-down	3+

Key Insight

Training only on one domain fails on the other.
Both videos must be in the training set.

Auto-Annotation with LocateAnything-3B

LocateAnything (NVLabs Eagle/Embodied)

- 3B-parameter vision-language model
- Open-vocabulary: detect any class by text
- Outputs normalised box tokens <box>x1 y1 x2 y2</box>
- No fine-tuning needed — zero-shot detection

Class prompts used

- "military tank"
- "armored vehicle"
- "battle tank"

8-bit quantisation

Loading with `load_in_8bit=True` reduces VRAM from

Annotation Script

```
python annotate_video_locate.py \  
  --video tank1.mp4 \  
  --classes "military_tank" \  
            "armored_vehicle" \  
            "battle_tank" \  
  --stride 1 \  
  --min-area 0.0001 \  
  --out dataset_tank1/
```

Output Format (YOLO)

```
# class cx cy w h  
0 0.572 0.645 0.237 0.294
```

Dataset Construction

Post-processing Steps

1 Class collapse

All 3 class IDs (military tank, armored vehicle, battle tank) → class 0

2 Second-stage NMS

Greedy IoU dedup with threshold 0.3.
Removes overlapping boxes from different prompts

3 Train/val split

85 % train / 15 % val, random seed

4 Multi-video merge

tank1 + tank4 combined for domain coverage

Dataset Statistics

Source	Frames	Boxes
tank1.mp4	350	350
tank4.mp4	302	653
After NMS	—	511
Train set	367	—
Val set	103	—
Total	470	—

Box Reduction

1908 → 511 boxes after cross-prompt NMS (−73% duplicates removed)

YOLOv8 Training Strategy

Model: YOLOv8n

- 3M parameters, 8.2 GFLOPs
- Smallest YOLOv8 — fast inference on laptop GPU
- Pre-trained on COCO, fine-tuned on tank dataset

Augmentation (heavy, to compensate small dataset)

- Mosaic 1.0, MixUp 0.1
- Rotation ± 10 , scale 0.5
- Horizontal flip 0.5
- HSV jitter (H: 0.015, S: 0.7, V: 0.4)

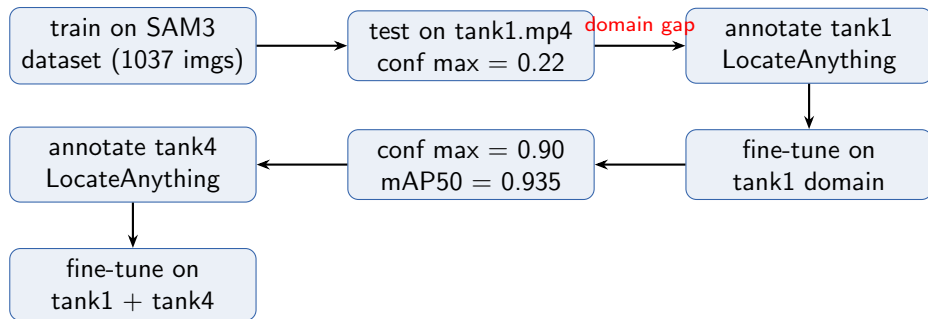
Training Configuration

Parameter	Value
Epochs	100 (early stop)
Image size	640×640
Batch size	16
Optimiser	AdamW (auto)
Patience	50
Device	RTX A3000 12 GB

Results

mAP50	0.935
mAP50-95	0.811
Precision	0.984
Recall	0.851

Iterative Domain Adaptation



Each annotation round adds domain coverage. The model progressively generalises from one video type to another.

FastMOT Architecture

FastMOT is a multi-object tracker that combines:

- 1 **Object Detector** — runs every N frames
- 2 **KLT Optical Flow** — tracks between detections
- 3 **Kalman Filter** — motion prediction
- 4 **Feature Extractor** — appearance ReID
- 5 **Hungarian Association** — detection $\leftrightarrow$ track matching

Per-frame Decision

Frame type	Operation
Frame 0	Full detect + init
Every N th frame	Detect + associate
Other frames	KLT flow only

Our modifications

Replaced TRT-only detector and ReID with PyTorch-native equivalents

Detector Integration: UltralyticsDetector

Problem: FastMOT's built-in YOLOv4Detector requires a TensorRT-compiled engine (YOLOv4 format).

Solution: New UltralyticsDetector class that:

- Wraps ultralytics.YOLO(best.pt)
- Implements FastMOT's async Detector ABC
- Runs inference in detect_async(), caches result
- Returns DET_DTYPE record array: (tlbr, label, conf)
- Includes second-stage greedy NMS ($\text{IoU} > 0.3$)

Detector Config (mot.json)

```
"detector_type": "ULTRALYTICS",  
"ultralytics_detector_cfg": {  
  "weights": "best.pt",  
  "conf_thresh": 0.35,  
  "iou_thresh": 0.25,  
  "min_aspect_ratio": 0.1,  
  "imgsz": 640  
}
```

Runtime

86 FPS on RTX A3000 12 GB
Detector frame skip = 5

ReID Integration: PyTorch Feature Extractor

Problem: FastMOT's OSNet ReID uses TensorRT 7 API, broken on TensorRT 10.

Solution: TorchFeatureExtractor using **MobileNetV3-Small** backbone:

- Pretrained on ImageNet (torchvision)
- Classifier head removed; 576-dim avgpool features
- L2-normalised embeddings
- Cosine distance for track association
- Graceful fallback chain: TRT → PyTorch → IoU-only

Feature Extractor

Property	Value
Backbone	MobileNetV3-S
Embedding dim	576
Distance metric	Cosine
Max ReID cost	0.85
Track max age	150 frames (≈ 26 s)

Startup log

```
[INFO] PyTorch ReID
(MobileNetV3) loaded
      appearance matching
      active
```

Other Compatibility Fixes

Issues fixed

- **GStreamer auto-detect**
WITH_GSTREAMER hardcoded True
→ now reads `cv2.getBuildInformation()`
- **Video codec**
avc1 requires HW encoder
→ fallback to mp4v, post-encode with libx264
- **numba/coverage conflict**
numba 0.65.1 needs coverage ≥ 7.0
- **Label map**
Class 0 showed as “head” (CrowdHuman)
→ pass `-l tank.names` at runtime

Wrapper Script (track_tank.sh)

```
#!/bin/bash
# Activate venv, run FastMOT,
# re-encode to H.264
python app.py \
  -i "$INPUT" \
  -o "$TMP" \
  -l tank.names \
  -m

ffmpeg -i "$TMP" \
  -vcodec libx264 \
  -crf 23 -preset fast \
  "$OUTPUT" -y
```

Detection & Tracking Results

Detection Performance

Metric	v1 (tank1)	v2 (merged)
mAP50	0.995	0.935
mAP50-95	0.872	0.811
Precision	0.999	0.984
Recall	1.000	0.851
Training imgs	323	470

v1 = overfits single domain; v2 = generalises across domains

Tracking Performance

Property	Value
Input resolution	1280×720
Detector skip	5 frames
Tracking FPS	86 (tank)
Confirm hits	3 frames
Duplicate thresh	IoU > 0.4
Max track age	150 frames
ReID metric	Cosine

Confidence Distribution on Test Videos

tank1.mp4: max=0.90, p75=0.72 tank4.mp4: max=0.69, p75=0.44

Pipeline Summary

Stage	Tool / Model	Key parameter
Frame extraction	OpenCV + FFmpeg	stride = 1
Auto-annotation	LocateAnything-3B (8-bit)	3 text prompts
NMS dedup	Greedy IoU	threshold = 0.3
Train/val split	random seed 42	85 % / 15 %
Detection model	YOLOv8n (3M params)	imgsz = 640
MOT framework	FastMOT	skip = 5
Detector in MOT	UltralyticsDetector	conf = 0.35
Between-frame	KLT Optical Flow	—
Motion model	Kalman Filter	—
Appearance ReID	MobileNetV3-Small (576-d)	cosine, cost ≤ 0.85
Output encoding	libx264 (FFmpeg)	CRF 23

Limitations & Future Work

Current Limitations

- Small training dataset (470 images)
- ReID struggles with partial visibility ($< 30\%$ in frame)
- MobileNetV3 not fine-tuned on tanks
→ generic ImageNet features
- No dedicated multi-camera support

Future Work

- **ReID fine-tuning**
Extract per-ID crops from tracked video, train with triplet/ArcFace loss on tank identities
- **More data**
Additional video domains, synthetic CARLA data
- **Larger detector**
YOLOv8s/m for better small-object recall
- **Depth integration**
Depth Anything v3 for 3-D track position

What was built

A fully automated pipeline from raw video to tracked military vehicle output:

- 1 **Zero-shot auto-annotation** via LocateAnything-3B (8-bit quantised, no GPU OOM)
- 2 **Domain-adaptive training** across aerial and side-view footage
- 3 **YOLOv8n** achieving $mAP50 = 0.935$ on merged dataset
- 4 **FastMOT** with YOLOv8 detector, KLT flow, Kalman filter, and PyTorch ReID running at **86 FPS**
- 5 **H.264 output** compatible with WhatsApp / mobile devices

Track any tank. Ship on WhatsApp.